Light

Programming Fundamentals

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Variables & constants
2. Data types & operators
3. Programming constructs
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Programming Fundamentals (J277/02 Section 2.2)**. This topic covers the essential building blocks of programming: variables, constants, data types, operators, inputs, outputs, and the three programming constructs (sequence, selection, iteration). You must be able to read, write, trace, and correct code in these areas.

Exam questions typically ask you to **write code** to solve problems, **trace** through existing code to determine outputs, **identify errors** in code snippets, and **explain** programming concepts. The 2024 exam included questions on iteration, string manipulation, and tracing through selection statements. Expect questions worth 2-8 marks requiring you to demonstrate practical programming skills.

Why programming fundamentals matter

Programming fundamentals are the foundation of all software development. Whether you're building a simple calculator or a complex game, you'll use variables to store data, operators to perform calculations, and control structures to make decisions and repeat actions. Mastering these concepts allows you to:

- **Store and manipulate data:** Variables and data types let you work with numbers, text, and logical values

- **Perform calculations:** Arithmetic and comparison operators enable mathematical operations and decision-making
- **Control program flow:** Selection and iteration allow programs to respond to conditions and repeat tasks
- **Interact with users:** Inputs and outputs enable communication between programs and users
- **Solve real problems:** Combining these elements lets you create solutions to practical challenges

  In the exam, you'll encounter code in OCR Exam Reference Language (pseudocode) and Python. You should be comfortable reading both and writing solutions in either format.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and demonstrate it in code.

- **Declare and use** variables and constants appropriately.
- **Explain** the difference between a variable and a constant.
- **Identify and use** the five main data types: integer, real, Boolean, character, string.
- **Use** arithmetic operators: +, -, *, /, MOD, DIV, ^ (exponentiation).
- **Use** comparison operators: ==, !=, <, >, <=, >=.
- **Use** Boolean operators: AND, OR, NOT.
- **Write** input and output statements to interact with users.
- **Write** assignment statements to store values in variables.
- **Implement** sequence, selection (IF-THEN-ELSE, nested IF, SWITCH/CASE), and iteration (FOR, WHILE, DO-UNTIL).
- **Trace** through code to determine variable values and outputs.
- **Identify and correct** syntax and logic errors in code.
- **Use** string manipulation techniques: length, substring, concatenation, case conversion.
- **Perform** basic file operations: open, read, write, close.
- **Generate** random numbers within a specified range.

> **Specification note.** OCR J277 Section 2.2 covers programming fundamentals. Questions will be set using OCR Exam Reference Language or a high-level language (typically Python). You must understand both pseudocode syntax and Python syntax. The exam reference language guide is provided in the exam, but familiarity with it beforehand is essential for speed.

Toolkit

Data types at a glance

Data type	Description	Examples	Python declaration
Integer	Whole numbers (positive, negative, or zero)	-5, 0, 42, 1000	age = 16

Data type	Description	Examples	Python declaration
Real (Float)	Numbers with decimal places	3.14, -0.5, 99.99	price = 19.99
Boolean	True or False values only	True, False	found = True
Character	A single letter, digit, or symbol	'A', '7', '!'	grade = 'A'
String	A sequence of characters (text)	"Hello", "ABC123"	name = "Alice"

Arithmetic operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	6 * 7	42
/	Division	15 / 4	3.75
`DIV` or `//`	Integer division (quotient)	15 DIV 4	3
`MOD` or `%`	Modulus (remainder)	15 MOD 4	3
^ or **	Exponentiation (power)	2 ^ 3	8

Comparison and Boolean operators

Comparison operators:

Operator	Meaning	Example
==	Equal to	x == 5
!= or <>	Not equal to	x != 5
<	Less than	x < 10
>	Greater than	x > 10
<=	Less than or equal	x <= 10

Operator	Meaning	Example
`>=`	Greater than or equal	x >= 10

Boolean operators:

Operator	Meaning	Example
`AND`	Both conditions must be true	x > 0 AND x < 10
`OR`	At least one condition must be true	x < 0 OR x > 100
`NOT`	Reverses the condition	NOT found

Boolean truth tables

Truth tables show the output for every possible combination of inputs. You must know these for the exam.

AND truth table:

A	B	A AND B
False	False	False
False	True	False
True	False	False
True	True	**True**

AND only returns True when BOTH inputs are True.

OR truth table:

A	B	A OR B
False	False	False
False	True	**True**
True	False	**True**
True	True	**True**

OR returns True when AT LEAST ONE input is True.

NOT truth table:

A	NOT A
False	**True**
True	False

NOT reverses the value — True becomes False and False becomes True.

> **Exam tip:** MOD and DIV are commonly tested. Remember: DIV gives the whole number of times the divisor fits (quotient), MOD gives what's left over (remainder). For 17 DIV 5 = 3 and 17 MOD 5 = 2, because 5 goes into 17 three times with 2 left over.

1. Variables and constants

Programs need to store and manipulate data. **Variables** and **constants** are named storage locations in memory that hold values during program execution.

1.1 Variables

A **variable** is a named location in memory whose value can change during program execution. Think of it as a labelled box that can hold different values at different times.

Key characteristics:

- Has a unique identifier (name)
- Stores a single value at any time
- Value can be changed during execution
- Has a specific data type

Declaring and assigning variables:

```
# Python examples
age = 16                # Integer variable
name = "Alice"          # String variable
price = 19.99           # Real/float variable
passed = True            # Boolean variable

# Changing a variable's value
age = 17                # age now holds 17
age = age + 1           # age now holds 18
```

OCR Exam Reference Language:

```
age = 16
name = "Alice"
price = 19.99
passed = true

age = 17
age = age + 1
```

1.2 Naming conventions

Good variable names make code easier to read and understand.

- **Use meaningful names:** `totalScore` not `ts`
- **Start with a letter:** `score1` is valid; `1score` is not
- **No spaces:** Use `firstName` or `first_name`
- **Case sensitive:** `Age` and `age` are different variables
- **Avoid reserved words:** Don't use `print`, `if`, `for` as names

> **Exam tip:** When asked to identify variables in code, look for names that have values assigned to them using `=`. Remember that the same variable can be assigned different values at different points in the program.

1.3 Constants

A **constant** is a named value that does not change during program execution. Once set, it remains the same throughout the program.

Key characteristics:

- Value is fixed when declared
- Cannot be changed during execution
- Often named in UPPERCASE by convention
- Used for values that should not change (e.g., VAT rate, pi)

Examples:

```
# OCR Exam Reference Language
const VAT_RATE = 0.20
const PI = 3.14159
const MAX_PLAYERS = 4

# Python (by convention - UPPERCASE)
VAT_RATE = 0.20
PI = 3.14159
MAX_PLAYERS = 4
```

1.4 Why use constants?

- **Prevents accidental changes:** Value cannot be modified by mistake
- **Easier maintenance:** Change the value once, and it updates everywhere
- **Self-documenting:** Name explains what the value represents
- **Avoids "magic numbers":** `VAT_RATE` is clearer than `0.20`

1.5 Variable vs constant comparison

Feature	Variable	Constant
Value can change?	Yes	No
When is value set?	Can be set/changed anytime	Set once at declaration
Typical use	Counters, user input, calculations	Fixed values like tax rates, limits
Naming convention	camelCase or snake_case	UPPERCASE_WITH_UNDERSCORES

> **Note:** Python doesn't have true constants — the UPPERCASE convention is just a signal to programmers not to change the value. In OCR pseudocode, use the `const` keyword to declare constants.

1.6 Inputs and outputs

Programs communicate with users through **inputs** (receiving data) and **outputs** (displaying results).

Input — receiving data from the user:

```
# Python
name = input("Enter your name: ")
age = int(input("Enter your age: "))
price = float(input("Enter price: "))

// OCR Exam Reference Language
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

Important: In Python, `input()` always returns a string. To get a number, you must convert it using `int()` for integers or `float()` for decimals.

Output — displaying results to the user:

```
# Python
print("Hello, World!")
print("Your age is:", age)
print("Name:", name, "Age:", age)

// OCR Exam Reference Language
print("Hello, World!")
print("Your age is: " + age)
```

Concatenation: Joining strings together using `+` or commas. When using `+`, all parts must be strings; convert numbers using `str()`.

Worked Example 1 — Variables and assignment

Question: What values do the variables hold after this code executes? [3 marks]

```
x = 5
y = 10
x = y
y = x + 3
```

Model answer:
Line 1: x = 5 **(1)**
Line 2: y = 10
Line 3: x = y, so x = 10 **(1)**
Line 4: y = x + 3 = 10 + 3 = 13 **(1)**
Final values: x = 10, y = 13

Exam tip: Trace through code line by line, updating values as you go. The order matters — line 4 uses the NEW value of x (10), not the original (5).

Worked Example 2 — Identifying variables and constants

Question: Explain the difference between a variable and a constant, and give an example of when each would be used. [4 marks]

Model answer:
A **variable** is a named storage location whose value can change during program execution **(1)**.
A **constant** is a named storage location whose value is fixed and cannot change once set **(1)**.
Variable example: A counter in a loop, such as counting how many guesses a user has made, because this needs to increase each time **(1)**.
Constant example: The value of pi (3.14159) in a calculation program, because this mathematical value never changes **(1)**.

2. Data types and operators

Every piece of data in a program has a **data type** that determines what operations can be performed on it and how it is stored in memory.

2.1 The five main data types

Integer: Whole numbers without decimal points. Used for counting, indexing, and whole-number calculations.

```
count = 42
temperature = -5
year = 2024
```

Real (Float): Numbers with decimal places. Used for measurements, prices, and precise calculations.

```
price = 19.99
pi = 3.14159
percentage = 75.5
```

Boolean: Only two possible values: True or False. Used for conditions, flags, and logical operations.

```
gameOver = False
loggedIn = True
found = False
```

Character: A single character (letter, digit, or symbol). In Python, characters are just strings of length 1.

```
grade = 'A'
initial = 'J'
symbol = '!'
```

String: A sequence of characters. Used for names, messages, and text data.

```
name = "Alice Smith"
message = "Hello, World!"
postcode = "AB12 3CD"
```

2.2 Type conversion (casting)

Sometimes you need to convert data from one type to another. This is called **casting**.

```
# String to integer
age_string = "16"
age_int = int(age_string)    # Result: 16

# String to float
price_string = "19.99"
price_float = float(price_string)  # Result: 19.99

# Number to string
score = 100
score_string = str(score)   # Result: "100"

# Float to integer (truncates)
decimal = 9.7
whole = int(decimal)       # Result: 9 (not rounded!)

# Integer to float
whole = 5
decimal = float(whole)     # Result: 5.0
```

> **Common mistake:** `int()` truncates (cuts off) decimal places — it does NOT round. So `int(9.9)` gives 9, not 10.

2.3 Using arithmetic operators

```
# Basic arithmetic
total = 10 + 5      # 15
diff = 10 - 5       # 5
product = 10 * 5    # 50
quotient = 10 / 4   # 2.5

# Integer division and modulus
whole = 10 // 4     # 2 (DIV)
remain = 10 % 4     # 2 (MOD)

# Exponentiation (power)
squared = 5 ** 2    # 25
cubed = 2 ** 3      # 8
```

2.4 MOD and DIV applications

These operators have many practical uses:

- **Check if even/odd:** `number MOD 2 == 0` means even
- **Extract digits:** `456 MOD 10 = 6` (last digit)
- **Wrap around:** Clock arithmetic, array cycling
- **Split into groups:** Converting seconds to minutes and seconds

```
# Convert 185 seconds to minutes and seconds
totalSeconds = 185
minutes = totalSeconds // 60    # 3 minutes
seconds = totalSeconds % 60     # 5 seconds
print(minutes, "minutes", seconds, "seconds")
```

Worked Example 3 — Using MOD and DIV

Question: Write a program that takes a number of seconds and converts it to minutes and remaining seconds. [4 marks]

Model answer:

```
totalSeconds = int(input("Enter seconds: "))
minutes = totalSeconds // 60
seconds = totalSeconds % 60
print("Minutes:", minutes)
print("Seconds:", seconds)
```

Marking: Correct input **(1)**, using DIV (//) for minutes **(1)**, using MOD (%) for remaining seconds **(1)**, correct output **(1)**.

Exam tip: For time conversions, DIV gives the larger unit (minutes from seconds) and MOD gives the remainder (leftover seconds).

Worked Example 4 — Boolean operators

Question: What is the output of the following code? [3 marks]

```
age = 15
hasMembership = True
if age >= 16 OR hasMembership == True:
    print("Access granted")
else:
    print("Access denied")
```

Model answer:
age >= 16 evaluates to False (because 15 is not >= 16) **(1)**
hasMembership == True evaluates to True **(1)**
False OR True evaluates to True, so "Access granted" is printed **(1)**

Exam tip: Remember OR only needs ONE condition to be true. AND needs BOTH to be true.

3. Programming constructs

All programs are built from three fundamental constructs: **sequence**, **selection**, and **iteration**. Mastering these allows you to create any algorithm.

3.1 Sequence

Sequence means instructions are executed one after another in the order they are written. This is the simplest construct.

```
# Instructions execute top to bottom
print("Step 1")
print("Step 2")
print("Step 3")

# Output:
# Step 1
# Step 2
# Step 3
```

3.2 Selection (IF-THEN-ELSE)

Selection allows the program to choose different paths based on conditions. The most common form is IF-THEN-ELSE.

```
# Simple IF
if age >= 18:
    print("You can vote")

# IF-ELSE
if score >= 50:
    print("Pass")
else:
    print("Fail")

# IF-ELIF-ELSE (multiple conditions)
if score >= 70:
    print("Merit")
elif score >= 50:
    print("Pass")
else:
    print("Fail")
```

OCR Exam Reference Language:

```
if score >= 70 then
    print("Merit")
elseif score >= 50 then
    print("Pass")
else
    print("Fail")
endif
```

3.3 Nested IF statements

IF statements can be placed inside other IF statements for more complex decisions.

```
if age >= 18:
    if hasDrivingLicence:
```

```
        print("You can rent a car")
    else:
        print("You need a licence")
else:
    print("You must be 18+")
```

> **Exam tip:** When tracing nested IF statements, check the OUTER condition first. Only if it's true do you evaluate the INNER conditions.

3.4 Iteration – FOR loops

FOR loops repeat a fixed number of times. Use when you know how many iterations are needed.

```
# Count from 1 to 5
for i in range(1, 6):
    print(i)
# Output: 1, 2, 3, 4, 5

# Loop through a string
for letter in "Hello":
    print(letter)
# Output: H, e, l, l, o

# Count down
for i in range(5, 0, -1):
    print(i)
# Output: 5, 4, 3, 2, 1
```

OCR Exam Reference Language:

```
for i = 1 to 5
    print(i)
next i
```

3.5 Iteration – WHILE loops

WHILE loops repeat while a condition is true. Use when you don't know how many iterations are needed in advance.

```
# Keep asking until valid input
password = ""
while password != "secret123":
    password = input("Enter password: ")
print("Access granted")

# Count up to a limit
count = 1
while count <= 5:
    print(count)
    count = count + 1
```

3.6 Iteration – DO-UNTIL loops

DO-UNTIL loops (also called REPEAT-UNTIL) execute at least once, then repeat until a condition becomes true.

```
// OCR Exam Reference Language
do
    guess = input("Enter guess: ")
until guess == correctAnswer

# Python equivalent (no built-in do-until)
while True:
    guess = input("Enter guess: ")
    if guess == correctAnswer:
        break
```

> **Key difference:** WHILE checks the condition BEFORE executing the loop body (might never run). DO-UNTIL checks AFTER (always runs at least once).

3.7 SWITCH/CASE statements

SWITCH/CASE provides an alternative to multiple IF-ELIF statements when checking a single variable against multiple values. This is primarily used in OCR pseudocode.

```
// OCR Exam Reference Language
switch day:
    case "Monday":
        print("Start of week")
    case "Friday":
        print("End of work week")
    case "Saturday":
        print("Weekend!")
    case "Sunday":
        print("Weekend!")
    default:
        print("Midweek day")
endswitch
```

Python equivalent using IF-ELIF:

```
if day == "Monday":
    print("Start of week")
elif day == "Friday":
    print("End of work week")
elif day == "Saturday" or day == "Sunday":
    print("Weekend!")
else:
    print("Midweek day")
```

> **Exam tip:** In the exam, SWITCH/CASE will appear in OCR pseudocode. In Python, you would use multiple IF-ELIF statements to achieve the same result.

3.8 Choosing the right loop

Different loops are suited for different situations:

Loop type	When to use	Example situation
FOR	Known number of iterations	Print numbers 1-10, process each item in a list
WHILE	Unknown iterations, might not run at all	Search for value, read until end of file
DO-UNTIL	Unknown iterations, must run at least once	Menu selection, input validation

3.9 String manipulation

Strings are sequences of characters that can be manipulated in various ways.

Common string operations:

```
text = "Hello World"

# Length
len(text)              # 11

# Accessing characters (0-indexed)
text[0]                 # 'H'
text[6]                 # 'W'
text[-1]                # 'd' (last character)

# Substring (slicing)
text[0:5]               # 'Hello'
text[6:11]              # 'World'
text[6:]                # 'World' (to end)

# Case conversion
text.upper()           # 'HELLO WORLD'
text.lower()           # 'hello world'
```

More string operations:

```
# Concatenation (joining)
first = "Hello"
second = "World"
combined = first + " " + second
# Result: "Hello World"

# Finding characters
text = "Hello"
text.find("l")         # 2 (first occurrence)

# Checking content
"H" in text            # True
"X" in text            # False

# OCR Exam Reference Language
text.length            // Length
```

```
text.substring(0,5)    // First 5 characters
text.left(5)           // First 5 characters
text.right(5)          // Last 5 characters
```

Exam tip: String indexing starts at 0, not 1. So in "Hello", index 0 is 'H', index 1 is 'e', etc. This is a very common source of errors.

ASCII conversion: OCR pseudocode includes `ASC()` to convert a character to its ASCII code number (A=65, a=97, 0=48) and `CHR()` to convert an ASCII code back to a character. In Python, these are `ord()` and `chr()`. This is useful for encryption, validation, and character manipulation.

```
# ASCII conversion in Python
ord('A')    # 65
ord('a')    # 97
ord('0')    # 48
chr(65)     # 'A'
chr(97)     # 'a'

// OCR Exam Reference Language
ASC("A")   // 65
CHR(65)    // 'A'
```

3.10 File handling

Programs can read from and write to files to store data permanently. This is essential when you need data to persist after the program ends, such as saving high scores, user preferences, or log files.

Reading from a file:

```
# Python - read entire file at once
file = open("data.txt", "r")
content = file.read()
file.close()

# Read line by line (better for large files)
file = open("data.txt", "r")
for line in file:
    print(line)
file.close()

# Read a single line
file = open("data.txt", "r")
firstLine = file.readline()
file.close()

// OCR Exam Reference Language
myFile = openRead("data.txt")
content = myFile.readLine()
myFile.close()
```

Writing to a file:

```
# Python - write to file (overwrites existing)
file = open("output.txt", "w")
file.write("Hello, World!\n")
file.write("Line 2\n")
file.close()

# Append to existing file (adds to end)
file = open("output.txt", "a")
file.write("New line added\n")
file.close()

// OCR Exam Reference Language
myFile = openWrite("output.txt")
myFile.writeLine("Hello, World!")
myFile.close()
```

File modes:

- `"r"` = Read mode — opens file for reading only; file must exist
- `"w"` = Write mode — creates new file or overwrites existing file
- `"a"` = Append mode — adds to end of file; creates file if it doesn't exist

> **Important:** Always close files when finished using `file.close()`. Failing to close files can lead to data loss or prevent other programs from accessing the file.

3.11 Random number generation

Random numbers are useful for games, simulations, testing, and any situation where unpredictability is needed. Python requires importing the `random` module before generating random numbers.

```
# Python - must import random module first
import random

# Random integer between 1 and 6 (inclusive) - like a dice
dice = random.randint(1, 6)

# Random integer between 1 and 100
number = random.randint(1, 100)

# Random float between 0 and 1
decimal = random.random()

# Random choice from a list
colours = ["red", "green", "blue"]
chosen = random.choice(colours)

# Shuffle a list randomly
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)

// OCR Exam Reference Language (simpler syntax)
dice = random(1, 6)
```

Exam tip: Remember that `random.randint(1, 6)` includes BOTH endpoints — so it can generate 1, 2, 3, 4, 5, OR 6. This is different from `range()` which excludes the end value.

Worked Example 5 — FOR loop

Question: Write a program that outputs the 7 times table from 1×7 to 12×7. [3 marks]

Model answer:

```
for i in range(1, 13):
    result = i * 7
    print(i, "x 7 =", result)
```

Marking: Correct loop range (1 to 12) **(1)**, correct multiplication **(1)**, appropriate output **(1)**.

Worked Example 6 — WHILE loop

Question: Write a program that keeps asking the user for a password until they enter "pass123". [4 marks]

Model answer:

```
password = ""
while password != "pass123":
    password = input("Enter password: ")
print("Access granted")
```

Marking: Initialise password **(1)**, correct WHILE condition **(1)**, input inside loop **(1)**, message after loop **(1)**.

Exam tip: When using WHILE, make sure to initialise the variable BEFORE the loop and update it INSIDE the loop, otherwise you'll create an infinite loop.

Worked Example 7 — String manipulation

Question: Write a program that takes a user's full name and outputs their initials. For example, "John Smith" should output "JS". [4 marks]

Model answer:

```
fullName = input("Enter full name: ")
spacePosition = fullName.find(" ")
firstInitial = fullName[0]
secondInitial = fullName[spacePosition + 1]
print(firstInitial + secondInitial)
```

Alternative solution:

```
fullName = input("Enter full name: ")
names = fullName.split(" ")
initials = names[0][0] + names[1][0]
print(initials)
```

Marking: Get input **(1)**, extract first initial **(1)**, extract second initial **(1)**, output initials **(1)**.

Worked Example 8 — Tracing through code

Question: What is the output of the following code? [4 marks]

```
total = 0
for i in range(1, 5):
    if i % 2 == 0:
        total = total + i
print(total)
```

Model answer:
i = 1: 1 % 2 = 1, not equal to 0, so total stays 0 **(1)**
i = 2: 2 % 2 = 0, condition true, total = 0 + 2 = 2 **(1)**
i = 3: 3 % 2 = 1, not equal to 0, so total stays 2
i = 4: 4 % 2 = 0, condition true, total = 2 + 4 = 6 **(1)**
Loop ends (range goes 1, 2, 3, 4 — not including 5)
Output: 6 (1)

Exam tip: Use a trace table to track variable values. Write down the value of each variable after each line executes.

Worked Example 9 — File handling

Question: Write a program that reads all the lines from a file called "names.txt" and prints each name on a new line. [3 marks]

Model answer:

```
file = open("names.txt", "r")
for line in file:
    print(line)
file.close()
```

Marking: Open file for reading **(1)**, loop through lines **(1)**, close file **(1)**.

Worked Example 10 – Random numbers

Question: Write a program that simulates rolling two dice and outputs the total. [3 marks]

Model answer:

```
import random
dice1 = random.randint(1, 6)
dice2 = random.randint(1, 6)
total = dice1 + dice2
print("Total:", total)
```

Marking: Import random **(1)**, generate two random numbers 1-6 **(1)**, calculate and output total **(1)**.

Worked Example 11 – Identifying and correcting errors

Question: The following code should output the average of three numbers entered by the user. Identify and correct the errors. [4 marks]

```
num1 = input("Enter first number: ")
num2 = input("Enter second number: ")
num3 = input("Enter third number: ")
average = num1 + num2 + num3 / 3
print("Average: " + average)
```

Model answer:

Error 1: Input returns strings, need to convert to numbers **(1)**

Error 2: Division happens before addition (operator precedence) — need brackets **(1)**

Error 3: Can't concatenate string with number — need to convert average to string **(1)**

Corrected code:

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
num3 = float(input("Enter third number: "))
```

```
average = (num1 + num2 + num3) / 3
print("Average: " + str(average))
```

(1) for correct final code

Worked Example 12 — Combined program

Question: Write a program that generates a random number between 1 and 100, then lets the user guess until they get it right. Tell them if each guess is too high or too low. [6 marks]

Model answer:

```
import random
target = random.randint(1, 100)
guess = 0

while guess != target:
    guess = int(input("Enter guess: "))
    if guess < target:
        print("Too low!")
    elif guess > target:
        print("Too high!")

print("Correct!")
```

Marking: Generate random number **(1)**, while loop with correct condition **(1)**, get user input **(1)**, check if too low **(1)**, check if too high **(1)**, correct message when finished **(1)**.

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on programming fundamentals. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	Tracing variable assignment	Line-by-line variable tracking
2	Variable vs constant explanation	Define + give contextual examples
3	MOD and DIV calculation	Time conversion problem
4	Boolean operator evaluation	OR logic with conditions
5	FOR loop writing	Times table output
6	WHILE loop writing	Password validation

Example	Question type	Key skill
7	String manipulation	Extract initials from name
8	Code tracing with loop	Sum even numbers
9	File handling	Read and display file contents
10	Random number generation	Dice simulation
11	Error identification and correction	Type conversion and precedence
12	Combined programming task	Number guessing game

Quick reference

Key terms — exam-ready definitions

Term	Definition (mark scheme style)
Variable	A named storage location in memory whose value can change during program execution
Constant	A named storage location whose value is fixed and cannot change once set
Data type	The kind of data a variable can hold (e.g., integer, string, Boolean)
Assignment	Storing a value in a variable using the = operator
Casting	Converting data from one type to another (e.g., string to integer)
Sequence	Instructions executed one after another in the order they are written
Selection	Choosing different paths based on conditions (IF-THEN-ELSE)
Iteration	Repeating instructions multiple times (FOR, WHILE, DO-UNTIL loops)
Concatenation	Joining strings together using the + operator
Index	The position of a character in a string (starting from 0)
Substring	A portion of a string extracted using slicing
Boolean operator	Operators that work with True/False values (AND, OR, NOT)

Python vs OCR pseudocode comparison

Operation	Python	OCR Exam Reference Language
Output	print("text")	print("text")
Input	input("prompt")	input("prompt")
Integer division	//	DIV
Modulus	%	MOD
Exponent	**	^
FOR loop	for i in range(1, 6):	for i = 1 to 5 ... next i
WHILE loop	while condition:	while condition ... endwhile
IF statement	if condition:	if condition then ... endif
String length	len(string)	string.length
Substring	string[0:5]	string.substring(0,5)
Random number	random.randint(1,6)	random(1,6)
Open file (read)	open("file.txt", "r")	openRead("file.txt")
Open file (write)	open("file.txt", "w")	openWrite("file.txt")

Common MOD and DIV results

Expression	Result	Explanation
17 DIV 5	3	5 fits into 17 three times
17 MOD 5	2	17 - (3 × 5) = 2 remainder
10 DIV 3	3	3 fits into 10 three times
10 MOD 3	1	10 - (3 × 3) = 1 remainder
25 DIV 7	3	7 fits into 25 three times
25 MOD 7	4	25 - (3 × 7) = 4 remainder
8 DIV 10	0	10 doesn't fit into 8
8 MOD 10	8	All 8 is remainder

Typical pitfalls

Pitfall 1 — Forgetting to convert input.

Wrong: `age = input("Age: ")` then doing `age + 1`
Right: `age = int(input("Age: "))` — input() returns a string!

Pitfall 2 — Confusing = and ==.
Wrong: `if x = 5:`
Right: `if x == 5:` — use = for assignment, == for comparison.

Pitfall 3 — Off-by-one errors in range().
Wrong: `range(1, 10)` thinking it gives 1-10
Right: `range(1, 11)` — range excludes the end value!

Pitfall 4 — String indexing starts at 0.
Wrong: `"Hello"[1]` returns 'H'
Right: `"Hello"[1]` returns 'e' — index 0 is 'H'.

Pitfall 5 — Infinite loops.
Wrong: Forgetting to update the loop variable inside WHILE.
Right: Always ensure the condition will eventually become false.

Pitfall 6 — Confusing MOD and DIV.
Wrong: Using MOD when you want the quotient.
Right: DIV = quotient (whole divisions), MOD = remainder.

Pitfall 7 — Operator precedence.
Wrong: `10 + 20 / 2` thinking it gives 15
Right: Division happens first: 20/2=10, then 10+10=20. Use `(10 + 20) / 2` to get 15.

Pitfall 8 — Not closing files.
Wrong: Opening a file but forgetting to close it.
Right: Always call `file.close()` when finished.

Pitfall 9 — Using int() on decimals.
Wrong: Thinking `int(9.9)` rounds to 10.
Right: `int()` truncates (cuts off) decimals — result is 9.

Pitfall 10 — Boolean logic errors.
Wrong: Using AND when OR is needed (or vice versa).
Right: AND = both must be true. OR = at least one must be true.

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.